

# A Safety-Bounded SDC-to-MCP Gateway for Medical AI Agents

Bennet Gerlach

Stefan Fischer

University of Lübeck, Lübeck, Germany  
`{be.gerlach,stefan.fischer}@uni-luebeck.de`

September 25, 2026

## Abstract

The Model Context Protocol (MCP) provides a common interface through which AI applications discover and use external resources and tools. It allows language-model agents to ground their reasoning in current system state and interact with heterogeneous services. In medical environments, however, exposing device state and action affordances requires deterministic constraints on possible effects. We present an IEEE 11073 Service-Oriented Device Connectivity (SDC)-to-MCP gateway that exposes metrics, alarms, context references, and semantic metadata as read-only resources, while representing selected action affordances as policy-validated dry-run tools. The term *safety-bounded* denotes a narrow no-execution property: agent-facing requests dispatch no SDC device operation. A Python prototype supports simulated fault and lifecycle experiments, a software-reference protocol path spanning independent Java and Python implementations, deterministic baselines, representation ablations, and multi-model agent evaluation. The results show semantically explicit resource exposure, visible rejection of invalid or outdated state, and preservation of the no-execution boundary across resource, proposal, and authorization paths. Explicit semantic metadata improved conformity to required metric identifiers in structured alarm outputs relative to a generic representation, while retained structured-output failures reveal a distinction between plausible narrative answers and task-compliant machine-readable results.

**Keywords:** IEEE 11073 SDC; Model Context Protocol; medical device interoperability; AI agents; safety boundaries.

## 1 Introduction

Operating rooms (ORs) and intensive care units (ICUs) increasingly depend on vendor-neutral device communication, traceable data exchange, and controlled access to clinical device state. IEEE 11073 Service-Oriented Device Connectivity (SDC) provides a service-oriented framework for medical device interoperability and has been studied for networked OR and ICU scenarios, including alarm distribution, isolation-room control, and robotic integration [1–4]. At the same time, large language model (LLM) agents are becoming attractive as tools also in medical environments, and they can dynamically access external data sources and tools, typically using the Model Context Protocol (MCP) [5, 6]. The combination is tempting: an agent could inspect connected point-of-care devices, summarize current alarms, and prepare structured proposals for clinical workflows. However, physical medical devices cannot be exposed to such agents like ordinary software APIs. The key challenge is to make device state legible to agents without granting non-deterministic models authority over device operation.

MCP exposes resources, prompts, and tools to AI applications through JSON-RPC, a structured remote-procedure-call interface [7, 8]. We study two deliberately constrained uses of an SDC-to-MCP gateway: (1) *resource-based situational awareness*, in which an agent reads device state and alarms without recommending treatment or parameter changes; and (2) a *dry-run action proposal*, in which the gateway validates structured arguments and then accepts

or rejects the dry-run while not allowing executions. Neither use case grants device-operation authority.

We ask four research questions: *RQ1*, can state obtained through a software-reference SDC protocol path be projected into stable, readable MCP resources with explicit semantics and provenance? *RQ2*, does every exposed MCP interaction preserve the no-execution invariant? *RQ3*, does the boundary fail visibly under unavailable, invalid, stale, delayed, or reordered state? *RQ4*, how accurately and consistently do agents interpret the resources relative to deterministic and representation baselines?

Accordingly, we contribute: (i) a deterministic projection of SDC device, metric, alarm, context, and operation-related constructs to read-only MCP resources and dry-run tools; (ii) an SDC-specific *Metadata-Interoperability-Exchange* (SDC-MIE), inspired by TogoMCP [9], that makes codes, handles, units, alarm references, provenance, and policy metadata explicit; and (iii) a reproducible prototype and evaluation framework for the resource, agent, fault, and proposal paths under the bounded invariant.

## 2 Background and Related Work

SDC represents device capabilities and state using the Basic Integrated Clinical Environment Protocol Specification (BICEPS) and its Medical Device Information Base (MDIB), which contains descriptors and states for components, metrics, alerts, operations, and context information [1]. An SDC provider exposes a device’s MDIB and services; an SDC consumer discovers the provider and accesses those services. Classical SDC systems support deterministic system-to-system interoperability, but do not define how such state should be exposed to LLM agents or how model-invoked actions should be constrained.

Standard-based operating-room integration provides the device-interoperability foundation for this work [10, 11]. The IHE Service-oriented Device Point-of-care Interoperability (SDPi) profiles build on this foundation with profiles for plug-and-trust connectivity, reporting, alerting, and external control [12]. SDPi is therefore part of the point-of-care device integration layer, not an electronic health record (EHR) interface. Our gateway consumes a restricted device-state view and adds an agent-facing representation; it does not replace SDPi or establish conformance to its profiles.

Fast Healthcare Interoperability Resources (FHIR), in contrast, provides resources and exchange mechanisms for healthcare information, including observations and device-related information [13]. FHIR/EHR integration and SDC-based device interaction can coexist. The distinction relevant here is the evaluated interaction contract: this gateway translates a current device-state snapshot into an agent-readable catalogue, rather than implementing a longitudinal clinical record or measuring the performance of a FHIR server.

MCP standardizes resources and tools but does not itself supply medical-device semantics or an application-specific no-execution policy [7, 8]. Healthcare MCP systems such as MCP-AI and TogoMCP address clinical reasoning or life-science knowledge access [9, 14]. Our contribution addresses a different interface: current SDC-derived device state, explicit mapping status, and a deterministic restriction on the effects of agent-facing requests. This is a scoped integration contribution, not a claim that all healthcare MCP systems are interchangeable or that generic MCP is intrinsically unsafe.

The proposed gateway is positioned between these layers. It uses SDC-like device-state structures as the provider-facing model, maps them into agent-readable MCP resources, and restricts action-related affordances to dry-run tools with explicit validation and no execution authority. Table 1 compares intended use, temporal semantics, agent interface, and write authority conceptually; it is not a measured ranking of substitutable systems.

This distinction matters because the same physiological value has different implications depending on how it is exposed. As an EHR observation, it becomes part of a longitudinal

Table 1: Conceptual positioning (not a performance ranking).

| Approach           | Intended/temporal semantics    | seman- | Agent interface            | Write authority/boundary        |
|--------------------|--------------------------------|--------|----------------------------|---------------------------------|
| FHIR/EHR           | Longitudinal record/work-flow  |        | Patient-data APIs          | Separate from device operations |
| Direct SDC         | Current MDIB/device state      |        | Application-specific       | SDC roles/local logic           |
| Generic MCP        | Application-defined context    |        | Generic resources/tools    | Server-specific                 |
| <b>SDC-MCP+MIE</b> | <b>Current state+semantics</b> |        | <b>Read-only/proposals</b> | <b>No-execution boundary</b>    |

patient record. As an SDC metric state, it is part of a live device model. As an MCP resource, it becomes context for an agent. As an MCP tool argument, it may become part of an action proposal. A safety-bounded gateway must therefore preserve the provenance, scope, and access semantics of the original device information while translating it into an interface that agents can inspect and reason over. Our design focuses on this boundary rather than on replacing deterministic SDC consumers or clinical information systems.

### 3 Architecture and Implementation

Figure 1 provides an architectural overview, including prospective extensions beyond the evaluated prototype. Blue dashed arrows depict device-state flow, black arrows denote control or protocol communication, and green dash-dotted arrows indicate a future controlled-write path that is not implemented or evaluated here. The patient monitor and ventilator represent the simulated device scenarios; the infusion pump illustrates a possible extension only. The device-facing side provides provider discovery, state extraction, and a normalized state view. The agent-facing side exposes MCP resources and, in dry-run configurations, selected MCP tools. A policy and audit layer separates proposal validation from any device-side execution.

The Python prototype uses the MCP software development kit (SDK) over standard input/output (stdio) and evaluates its `sdc11073` consumer against separate Python providers and the independent Java SDC reference implementation (SDCri) 7.0.0. Both paths exercise Transport Layer Security (TLS), Simple Object Access Protocol (SOAP), Extensible Markup Language (XML) parsing, and MDIB initialization. Stdio confines the MCP boundary to a same-host subprocess; no network-facing MCP listener is evaluated. The evaluated configuration is intentionally conservative: resources are read-only; dry-run tools can validate proposals and write audit-ready results, but no SDC Set Service or ActivateOperation call is executed. Any approval is recorded without dispatching a device operation; the figure’s reference to approved actions describes the prospective architecture, not execution authority in this prototype.

The bounded property concerns device effects, not general safety. In controlled tests without external updates,  $\mathcal{Q}_{\text{MCP}}$  denotes exposed requests and  $\mathcal{V}$  observed devices.  $W(q)$  counts SDC Set Service or ActivateOperation calls triggered by  $q$ ;  $\mathcal{S}_d^{\text{pre}}(q)$  and  $\mathcal{S}_d^{\text{post}}(q)$  are  $d$ ’s normalized states before and after  $q$ . We test

$$\forall q \in \mathcal{Q}_{\text{MCP}}, \forall d \in \mathcal{V} : W(q) = 0 \wedge \mathcal{S}_d^{\text{post}}(q) = \mathcal{S}_d^{\text{pre}}(q).$$

Resources are passive; tools validate only dry runs. We assume uncompromised code/configuration, an observation-only adapter, and no process-boundary bypass. Compromised hosts, forged providers, malicious configuration, authentication failures, and clinical appropriateness are excluded; deployment requires a separate safety case.

The two parts of this check have different interpretations. Zero dispatched operations is the intended request-level boundary. Equal pre-/post-request state digests are an additional observation in a controlled test without independent device updates; a real provider may

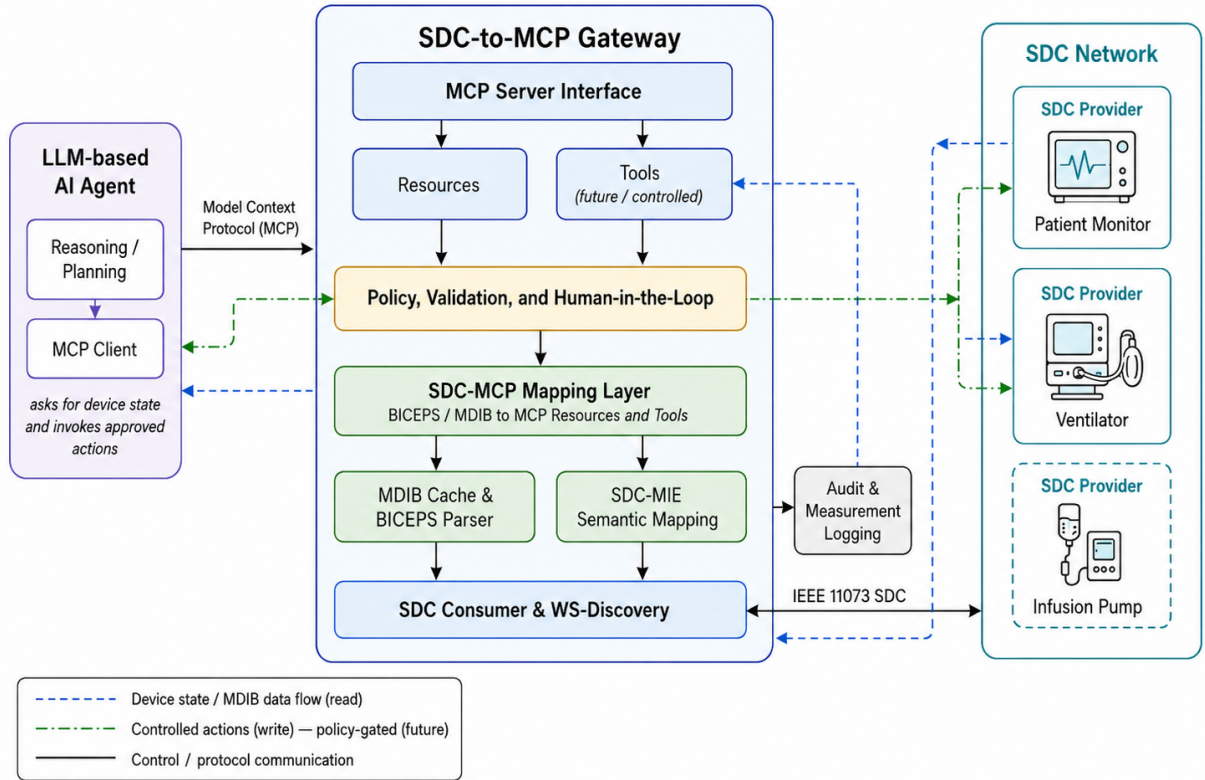


Figure 1: SDC-to-MCP gateway architecture and intended extension points. The evaluated prototype exposes read-only MCP resources and policy-checked dry-run tools; proposals and approvals do not execute device operations. Green write-path arrows and the “future/controlled” tool label denote prospective functionality only. The infusion pump is an illustrative extension, not an evaluated provider.

Table 2: Responsibility and accountability boundary.

| Actor           | Responsibility; excluded authority                                    |
|-----------------|-----------------------------------------------------------------------|
| Agent           | Interpret context/propose; no clinical or device authority            |
| MCP host/client | Launch/constrain subprocess and protect credentials; no policy bypass |
| Gateway         | Map, validate, redact, audit, and enforce no execution                |
| SDC provider    | Supply identity/state; authenticity not established by EPR alone      |
| Clinician       | Retain clinical judgment and any future approval                      |
| Operator        | Approve configuration, protect host/keys/logs, anchor audits          |

legitimately change its measurements while a read request is being processed. Such an external change would not by itself indicate gateway write authority. The equality check is therefore not asserted for an arbitrary live clinical network, and the tests do not prove that all possible executions of the implementation satisfy the property.

The implementation enforces this boundary at independent layers: configuration loading rejects write-enabled modes; the SDC consumer contract exposes only discovery and snapshot reads; tool policies and result models require dry-run, non-executing semantics; and boundary tests wrap the adapter with a Set Service/ActivateOperation attempt counter.

Our threat model protects device-state integrity, configuration provenance, credentials, and audit evidence against malformed MCP requests, prompt-like resource content, look-alike provider identifiers, configuration changes, and post-hoc log edits, assuming uncompromised code, host, and process boundary. Exact endpoint reference (EPR) allowlisting—matching the protocol-level identifier of an SDC provider—fail-closed schemas, recursive secret redaction, and configuration/snapshot hashes address this local scope. Exact strings do not authenticate providers, and a compromised host remains out of scope. A future network-facing deployment would require mutual authentication, least-privilege resource/tool authorization, managed TLS, replay/rate/size controls, and externally anchored audit retention.

Transport security and application authority are separate. Mutual Transport Layer Security (mTLS) in the software-SDC testbeds authenticates the configured TLS peers; it does not authenticate the free-form identity fields in the synthetic approval workflow. Likewise, an EPR allowlist restricts accepted identifiers but is not a cryptographic proof of provider identity. The local MCP transport does not evaluate remote user authentication or multi-tenant authorization. These distinctions prevent successful protocol tests from being interpreted as a complete deployment-security assessment.

## 4 SDC-to-MCP Mapping

The gateway performs a structure-preserving projection of SDC device state into MCP primitives. It separates four concerns: extracting BICEPS/MDIB-facing constructs, normalizing them into a point-in-time device-state view, enriching them through SDC-MIE, and exposing them as MCP resources or policy-validated dry-run tools. We use *normalized device-state snapshot* for this point-in-time view. In a real SDC backend, it is derived from the provider’s MDIB; in the simulated backend, it is an MDIB-like normalized structure. The artifact evaluates both point-in-time snapshots and deterministic ordered-update states, but not a production SDC subscription stream.

Let the normalized device-state snapshot  $\mathcal{S} = (\mathcal{D}, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{O})$  denote descriptors, metric states, alarm states, context states, and operation-like affordances. The gateway defines the deterministic projection

$$\Phi(\mathcal{S}) = (\mathcal{R}, \mathcal{T}_{dry}), \quad (1)$$

where  $\mathcal{R}$  are read-only MCP resources and  $\mathcal{T}_{dry}$  are dry-run MCP tools derived only from explicitly whitelisted affordances. Resources expose device catalogues, mapped metrics, alarms,

Table 3: Conceptual SDC-to-MCP mapping.

| SDC/BICEPS-side construct       | MCP representation                       | Role               |
|---------------------------------|------------------------------------------|--------------------|
| Device/component descriptor     | <code>sdc://devices</code>               | Inventory          |
| Metric descriptor/state         | <code>sdc://devices/{id}/metrics</code>  | Measurements       |
| Alert condition/state           | <code>sdc://devices/{id}/alarms</code>   | Alarm status       |
| Context state                   | <code>sdc://devices/{id}/context</code>  | References         |
| Normalized state view           | <code>sdc://devices/{id}/mdib/raw</code> | Trace/debug        |
| Codes, handles, units, policies | <code>sdc://mapping</code>               | SDC-MIE metadata   |
| Operation affordance            | Dry-run MCP tool                         | Validated proposal |

```

schema_version: "1.0"
version: "1.0.0"
provenance: {source: "SDC-MCP research mapping"}
mappings:
- code: "150456"
  handles: ["metric.hr"]
  semantic_name: "heart_rate"
  label: "Heart rate"
  unit: "beats/min"
  access: "read-only"
  safety_class: "informational"
  requires_human_approval: false

```

Listing 1: Example extraction of the SDC-MIE YAML mapping artifact defining the heart\_rate metric.

context references, a trace/debug representation, and the mapping artifact; dry-run tools validate proposals but never execute SDC operations. Table 3 summarizes the projection.

Metric mapping extracts a device identifier, handle, nomenclature code, value, unit, and timestamp, then enriches this tuple with an agent-facing semantic name, label, description, access mode, and safety class. Alarm mapping first normalizes the SDC-facing alert state into alarm handle, kind, priority, presence, timestamp, and an optional related metric reference. It then links the alarm to the corresponding mapped metric so that an agent can associate, for example, a high airway-pressure alarm with the `airway_pressure` metric. Context mapping is intentionally conservative: patient, location, operator, and workflow states are exposed as references rather than expanded clinical records. This avoids conflating device-state access with EHR integration.

SDC-MIE is the gateway’s versioned, machine-readable semantic mapping document. It links SDC/BICEPS codes, handles, and units to agent-readable metric/alarm semantics, constraints, policies, and provenance. During  $\Phi$ , each tuple resolves as **mapped**, **unmapped**, **unsupported**, or **conflicting**; only mapped tuples are enriched. Formally,  $\mathcal{E}_{\text{MIE}} = (v_s, v_d, P, \mathcal{I})$  contains schema/-document versions, provenance, and entries. Its JSON Schema defines fields, access/safety classes, bounds, and approval metadata; cross-entry checks enforce unique identifiers and consistent units. Inspired by TogoMCP, SDC-MIE remains a research specification, not an IEEE 11073 standard or TogoMCP-compatible format. Listing 1 is an excerpt; the schema and examples reside in the artifact.

SDC service-control objects are not exposed directly. Whitelisted action-like tools perform schema, range, target, freshness, and policy checks and return only dry-run results (Listing 2). A

```

{
  "tool": "prepare_set_fio2",
  "device_id": "sim-ventilator-1",
  "operation": "set_fio2",
  "requested_value": 45.0,
  "allowed_range": [21.0, 100.0],
  "status": "accepted_dry_run",
  "reason": "policy_validated_no_execution",
  "requires_human_approval": true,
  "executed": false,
  "write_operations_allowed": false
}

```

Listing 2: Example of a policy-validated dry-run tool response for setting FiO<sub>2</sub> to 45%.

successful validation can enter a non-executing lifecycle through **proposed**, **policy\_validated**, and **pending\_approval**, followed by **approved**, **denied**, or **expired**. Each proposal binds device, operation, parameters, policy version, MDIB version, freshness, canonical snapshot hash, proposer, and expiry. Approval records an identity/reference only after repeating policy and current-state checks; changed, stale, or expired state cannot be overridden. Even **approved** requires **executed=false** and grants no SDC authority.

The mapping is deterministic for fixed snapshot, mapping, and policy inputs. For each provider, the gateway obtains the normalized state view, applies the SDC-MIE entries to metrics and alarms, builds device-scoped resource objects, and constructs the MCP resource catalogue. An unmapped code is not silently interpreted as a known medical concept. Its mapping status remains explicit, allowing tasks to require abstention rather than inference from an unfamiliar identifier. The original functional scenarios use supported metric mappings; the hold-out and protocol experiments deliberately include unmapped data.

#### 4.1 Interpreting a Mapped Metric

In the snapshot tuple  $\mathcal{S}$ , the five components are sets of descriptors, metric states, alarm states, context states, and operation-related affordances, respectively. The function  $\Phi$  is the agent-interface projection: it constructs readable resources and the limited proposal vocabulary from these inputs. In the semantic document  $\mathcal{E}_{\text{MIE}}$ ,  $v_s$  is the schema version,  $v_d$  the mapping-document version,  $P$  its provenance, and  $\mathcal{I}$  its mapping entries. These are configuration data, not instructions generated by the model.

#### 4.2 Worked Example: From Monitor State to Agent Response

Table 4 follows one metric through the multiple-device hold-out configuration. At the configured simulation time of 100 s, **holdout-monitor-a** reports a heart rate of 148 beats/min. Its medium-priority alarm is present because the scenario’s high threshold is 125 beats/min. These values describe a synthetic test fixture, not clinical thresholds proposed by the gateway. A second device simultaneously reports high airway pressure; retaining device and metric associations is therefore part of the task.

The mapper resolves code 150456 and handle **metric.hr** to the entry in Listing 1, verifies unit compatibility, and attaches the canonical name **heart\_rate**. The metric value remains source data: the mapping neither estimates nor alters it. The alarm resource retains the related metric handle. When constructing the serialized agent context, the evaluation harness joins active alarms to mapped metrics by device and handle, yielding an explicit metric name and value rather than requiring an inference from an alarm label. This deterministic context assembly is separate from the model’s interpretation.

Table 4: Trace of one synthetic monitor metric through the evaluated interface. Fields are selected for explanation, not a complete protocol message. The final row summarizes an archived GPT-4.1-mini response, repetition 1.

| Stage              | Selected content and role                                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Normalized source  | Device <code>holdout-monitor-a</code> ; handle <code>metric.hr</code> ; code <code>150456</code> ; value <code>148</code> ; unit <code>beats/min</code> . Alarm <code>alarm.hr.high</code> refers to this handle, is present, and has medium priority.                     |
| Declared semantics | SDC-MIE schema 1.0, document 1.0.0: <code>semantic_name=heart_rate</code> , read-only access, informational safety class. Compatible code/handle/unit yield <code>mapping_state=mapped</code> .                                                                            |
| MCP resources      | <code>sd://devices/holdout-monitor-a/metrics</code> carries the mapped measurement; the corresponding <code>/alarms</code> resource carries alarm state and its metric reference. Payloads include mapping version and source-file hash.                                   |
| Agent context      | A joined active-alarm record identifies the same device, <code>heart_rate</code> , value <code>148</code> , unit <code>beats/min</code> , and medium priority. The ventilator alarm remains a separate record.                                                             |
| Observed response  | The enriched-context model returned <code>active_alarm=true</code> , the monitor’s <code>heart_rate</code> alarm with medium priority, and the ventilator’s <code>airway_pressure</code> alarm with high priority. The frozen alarm-detection grader passed this response. |

**State quality is independent of mapping success.** In the separate invalid/stale hold-out configuration, a heart-rate value of 74 has handle `metric.hr.stale` and the same known code, but is explicitly stale. The oxygen-saturation value of 98 is explicitly invalid. Mapping can still identify these quantities; it must not turn them into current, trustworthy observations. In the archived first GPT-4.1-mini repetition, the freshness and validity tasks correctly returned `metric.hr.stale` and `metric.spo2.invalid`, respectively. Readable state and usable state are therefore distinct interface concepts.

The deterministic lifecycle suite tests the proposal boundary separately: a cached sample older than its configured 5 s threshold rejects a proposal with `stale_snapshot`; a sensor-validity fault rejects it with `invalid_snapshot`. These are separate controlled cases, not a continuous run of the monitor example or evidence that an LLM enforces policy. The  $\text{FiO}_2$  proposal in Listing 2 likewise illustrates the tool contract, not a treatment inferred from a heart-rate alarm. In every case, the gateway retains responsibility for validation and the absence of device execution.

Schema validation checks the shape of the mapping document; cross-entry validation checks relationships such as identifier uniqueness and unit consistency. Neither establishes that a manually supplied clinical label is medically correct. Mapping curation remains a separate responsibility, and the explicit document versions and hashes make the particular mapping used in a result inspectable.

## 5 Evaluation Methods

The evaluation separates representation correctness, protocol interoperability, device effects, and model behavior. Table 5 connects these evidence layers to the research questions. The same software project supports all layers, but their experimental units and acquisition paths differ: ordered simulator states are not a continuous SDC report stream, and hosted-model responses are not measurements from physical devices.

### 5.1 Simulated Devices and Task Context

The simulator represents a patient monitor and a ventilator. The monitor provides heart rate, peripheral oxygen saturation ( $\text{SpO}_2$ ), and respiratory rate, with high-heart-rate and low- $\text{SpO}_2$

Table 5: Research questions, evidence, and scope of inference.

| RQ | Evidence                                                                             | Tested outcome                                                              | Boundary                                                   |
|----|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------|
| 1  | Functional scenarios; Python and Java software providers                             | Readable resources, protocol snapshots, explicit mapping coverage           | No hardware or multi-vendor deployment                     |
| 2  | Instrumented requests, static checks, abstract workflow, proposal and approval cases | No attempted device operation; unchanged state in controlled tests          | Mechanically checked, not full implementation verification |
| 3  | Fourteen ordered-event cases                                                         | Visible rejection and recovery under specified faults and alarm transitions | No continuous subscription or clinical timing validation   |
| 4  | Twenty-eight task–scenario pairs, five models, representation ablation               | Task compliance, repeated-output consistency, interpretable failure modes   | Synthetic tasks; ablation uses one model                   |

Table 6: Synthetic hold-out configurations. Counts are distinct applicable task–scenario pairs before model repetitions.

| Configuration               | Main challenge                                                                    | Tasks |
|-----------------------------|-----------------------------------------------------------------------------------|-------|
| Multiple devices and alarms | Interpret simultaneous alarms and retain device associations                      | 6     |
| Ambiguous device catalogue  | Select exact same-type device, metric, and resource; resist embedded instructions | 6     |
| Unmapped metric             | Abstain from inventing the meaning of an unsupported code                         | 5     |
| Unavailable provider        | Identify missing state without inventing a device or URI                          | 5     |
| Invalid and stale state     | Distinguish validity and freshness from the presence of a value                   | 6     |

alarms. The ventilator provides inspired oxygen concentration ( $\text{FiO}_2$ ), positive end-expiratory pressure (PEEP), and airway pressure, with a high-airway-pressure alarm. These are synthetic device-state examples, not patient recordings or a validated physiological model.

Four functional scenarios establish basic resource behavior: a no-alarm baseline containing both devices, and separate tachycardia,  $\text{SpO}_2$ -drop, and high-airway-pressure scenarios. The combined baseline checks inventory and resource selection across devices; the single-device alarm scenarios give unambiguous expected states. Every advertised resource is read, and the resulting catalogue, metric tuples, alarm states, and resource uniform resource identifiers (URIs) are checked against scenario expectations. The catalogue contains shared health, discovery, device-list, and mapping resources, plus device-specific metrics, alarms, context, and normalized trace/debug views.

The final agent hold-out is separate from the four development scenarios. It contains five configurations with 28 applicable task–scenario pairs (Table 6). Four exploratory scenarios and their 16 development tasks were used to develop prompts and graders and are excluded from final model aggregates. Expected answers are supplied only to the grader. Task parameters, such as a requested device identifier, are part of the input and are not ground-truth answers.

## 5.2 Software-SDC Protocol Paths

The same-stack protocol experiment uses `sdc11073` 2.4.1 with provider and consumer in separate Linux containers and network namespaces on one host, with distinct IPv4 addresses. Three versioned Medical Device Information Base (MDIB) profiles are each exercised five times. The path covers WS-Discovery, mTLS, Simple Object Access Protocol (SOAP) `GetMdiB`, XML parsing, MDIB initialization, normalization, and MCP resource reads. WS-Discovery is the network discovery mechanism; an XAddr is a provider’s transport address. A directed-XAddr fallback would bypass discovery by supplying this address directly and is recorded separately

from discovery success.

An independent implementation check pairs the Python consumer with an unmodified SDCri 7.0.0 Java example provider. Five repetitions use loopback networking and separate TLS identities; an additional negative test attempts access with an untrusted client. This checks a cross-stack protocol path, not a multi-vendor medical-device installation. We report discovery, snapshot acquisition, resource readability, and mapping coverage separately: a syntactically valid metric with an unknown code must not become a falsely recognized clinical quantity.

Protocol latency is a local snapshot-to-resource measurement for the tested software path. Historical in-memory simulator timings exclude networking and are not pooled with SOAP/XML/TLS measurements. No experiment establishes a clinical deadline or a real-time guarantee.

### 5.3 No-execution, Lifecycle, and Provenance Checks

The no-execution test wraps the adapter with an operation-attempt counter and compares normalized device-state digests before and after requests. It exercises the advertised resources and valid, invalid, and unknown tool calls. Complementary checks inspect agent-facing source files for forbidden write calls and direct SDC imports, use property-based inputs for malformed arguments and event sequences, and traverse an abstract request/proposal workflow. The abstract workflow and source checks address different failure modes and do not constitute a formal proof of the Python implementation.

Fourteen deterministic ordered-event cases address unavailable, invalid, stale, delayed, re-ordered, and recovered state. They expose source/reception timestamps, age of information, provider availability, and sequence/MDIB versions. Alarm cases include simultaneous alarms, escalation, latching, acknowledgement, and suppression/reactivation. The harness supplies controlled state transitions rather than continuously subscribing to a live provider.

Seven dry-run policy cases cover FiO<sub>2</sub>, PEEP, and alarm-acknowledgement proposals. Seven additional authorization cases cover approval, denial, expiry, duplicate approval, stale state, changed state, and missing context. A decision rechecks policy, expiry, and snapshot binding, while leaving operation authority absent. These cases evaluate software behavior, not a clinician’s judgment or workflow usability.

The provenance/security experiment uses local standard input/output (stdio), a deterministic policy processor, synthetic credentials, exact and look-alike provider identifiers, and a chained audit file. It checks exact matching, argument rejection, redaction, provenance capture, chain verification, and detection of a modified record. No model call or remote MCP transport is involved in this experiment.

### 5.4 Agent Evaluation and Representation Ablation

A deterministic resource processor receives the same serialized MCP context as the language models and runs once per task–scenario pair. It does not receive the grader’s expected answers. Each of five hosted model endpoints runs all 28 pairs three times at temperature zero, yielding 84 cases per model and 420 external-model cases. Table 7 records the configured identifiers and hosting routes. A configured identifier is not necessarily an immutable weights snapshot: only the OpenAI identifier here contains an explicit snapshot date. No random seed was specified, and hosted services may change behind aliases.

The tasks cover inventory, alarm detection and alarm sets, constrained summaries without treatment or parameter advice, exact device/metric/URI selection, freshness, validity, availability, abstention on unknown mappings, and instruction/boundary refusal. Grading evaluates the requested structured response and its consistency with the scenario. For alarm-detection and alarm-set tasks, metric identifiers are compared exactly: `heart_rate` is not interchangeable with a prose label such as “High Heart Rate” or “heart rate.” For the summary task, an explicit Boolean alarm flag takes precedence over narrative alarm wording. A failed self-reported flag is

Table 7: Configured model identifiers in the frozen run. Hosting routes are experimental context, not privacy or quality endorsements.

| Model label      | Configured identifier                | Hosting route                  |
|------------------|--------------------------------------|--------------------------------|
| GPT-4.1 mini     | <code>gpt-4.1-mini-2025-04-14</code> | OpenAI                         |
| Gemini 2.5 Flash | <code>gemini-2.5-flash</code>        | Google Gemini API              |
| GPT-OSS 20B      | <code>gpt-oss:20b</code>             | Ollama Cloud                   |
| Gemma 4 26B      | <code>gemma-4-26b-a4b-it</code>      | University of Lübeck AI Lab    |
| Qwen 3.6 27B     | <code>qwen3.6-27b</code>             | Digital Hub Schleswig-Holstein |

Table 8: Recorded experimental provenance and replication units. Dates are in 2026. Reused reports are not additional model observations; snapshot reads are not independent devices.

| Evidence layer           | Date           | Unit and extent                                                                                              | Version or provenance anchor                                                                 |
|--------------------------|----------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| In-memory reference      | refer- 21 May  | Four scenarios, three runs each; 200 measured iterations after 20 warm-ups per run                           | Historical simulator aggregates; separate from protocol timings                              |
| Multi-model hold-out     | hold- 8 August | Five models $\times$ 28 task-scenario pairs $\times$ three repetitions; deterministic baseline once per pair | Frozen 32-file scientific input set; model identifiers in Table 7                            |
| Representation ablation  | 9 August       | Two new arms $\times$ 28 pairs $\times$ three repetitions; reuse 84 enriched and 28 deterministic cases      | Separate input lock; same GPT-4.1-mini snapshot and task definitions                         |
| Python/Python protocol   | 10 August      | Three MDIB profiles $\times$ five discovery/snapshot repetitions; eight MCP reads per snapshot               | <code>sdc11073 2.4.1</code> ; isolated containers, Python 3.12                               |
| Java/Python protocol     | 10 August      | One independent provider; five snapshot repetitions; eight MCP reads per snapshot                            | <code>SDCri 7.0.0 (cc46b3b)</code> ; <code>sdc11073 2.4.1</code>                             |
| Deterministic solidation | con- 10 August | 28 agent cases; 14 lifecycle cases; seven authorization cases; no-execution and mapping checks               | Clean code commit <code>7b269c0</code> ; model and protocol artifacts reused and hash-linked |

retained even when the surrounding answer appears benign; reported task failures are therefore not interchangeable with demonstrated unsafe actions or clinical misinterpretation.

The representation ablation uses GPT-4.1 mini on the same 28 pairs and three repetitions. The arms expose (i) raw normalized SDC tuples, (ii) generic MCP resources with no SDC-MIE enrichment, and (iii) enriched MCP resources. Device/metric facts are matched, while addressing structure and semantic metadata differ intentionally. The enriched arm reuses its archived model reports; the two other arms were generated after a separate input lock. URI-selection tasks are retained, so the raw comparison measures the combined contribution of catalogue structure and semantics, not semantic enrichment alone.

## 5.5 Analysis and Reproducibility Discipline

Table 8 distinguishes the acquisition paths and units of replication. The five-model hold-out was run on 8 August 2026. The raw and generic ablation arms were run on 9 August, reusing the enriched and deterministic reports from 8 August. Software-reference protocol evidence and the consolidated deterministic bundle are dated 10 August. Thus, the representation comparison holds scenarios and tasks fixed but is not a simultaneous, interleaved experiment across arms.

Frozen inputs include prompts, graders, mappings, policies, scenarios, and evaluation code.

Table 9: Software-reference protocol results. Mapping counts refer to exposed metric coverage, not multiplied repetition counts.

| Provider/consumer path        | Snapshots | MCP reads | Coverage and setting                           |
|-------------------------------|-----------|-----------|------------------------------------------------|
| Python/Python, three profiles | 15/15     | 120/120   | 7/11 metrics mapped; separate Linux containers |
| SDCri Java/Python             | 5/5       | 40/40     | 0/11 generic codes mapped; loopback            |

The 32-file scientific lock is distinct from the 34-file executed manifest, which additionally records the evaluation configuration and scientific-lock file. Their full SHA-256 digests are retained in the artifact. The hosted-agent runtime records Python 3.14.4, MCP SDK 1.27.1, Pydantic 2.13.4, and gateway package metadata 0.13.0; these runtime fields are not the later release-candidate tag. The public tag `v2.0.0-rc1` resolves to `4cf508e`, which adds the consolidated results to the tested code state. These identifiers distinguish tested code, recorded environment, and archival release.

Consolidation checks the recorded input-lock identity and reuses model outputs; it is not an independent replication of the hosted calls. A corrected ablation analysis reconstructed valid URI catalogues from frozen scenarios without changing answers or pass/fail grades. The individual reports preserve task identifiers, expected and observed fields, checks, and available call metadata, enabling inspection of a reported failure without contacting a hosted endpoint again.

We report observed counts and rates as descriptive results for this finite task suite. Three repetitions probe stability but are not three independent task samples, and the five configurations do not represent a random sample of clinical settings. We therefore do not use pooled binomial intervals or repetition-level significance tests to support population-level reliability claims. Paired improvements are also described at the task–scenario level. Latencies include endpoint and service conditions and are not controlled comparisons of model throughput. Monetary cost was not measured.

## 6 Results

### 6.1 Resource Exposure and Protocol Interoperability

All advertised resources were readable in the four functional scenarios. The two-device baseline exposed twelve resources, six metrics, and three alarm states. Each single-device alarm scenario exposed eight resources, three metrics, and its expected active alarm. All metric concepts required by these basic scenarios were mapped, and no resource-read error occurred.

Table 9 separates protocol success from semantic coverage. The same-stack container experiment completed 15/15 discovery attempts without directed-XAddr fallback, 15/15 MDIB snapshots, and 120/120 MCP resource reads. Seven of eleven metrics mapped; four unknown codes remained explicit. Unsupported descriptor/state classes were also reported rather than silently converted. Median snapshot-to-resource latency across profiles was 1.288 s–1.296 s.

The independent SDCri path completed WS-Discovery without fallback, 5/5 snapshots, and 40/40 resource reads. Mutual TLS with separate identities succeeded, while the untrusted-client test was rejected. The example provider’s generic codes mapped 0/11: this is successful conservative handling of unsupported semantics, not demonstrated semantic interoperability for those quantities. Median snapshot latency was 1.096 s. Both sets of timings characterize a local software testbed, not clinical-network performance.

Historical simulator-only measurements had medians of 4.88 ms–7.18 ms. These values exclude the protocol path and serve as local software measurements; their difference from the protocol timings is not a measured component-by-component latency decomposition.

Table 10: Descriptive model and representation results. Repeated cases are not independent task samples. Mean hosted-call times include service conditions.

| Model or representation                                                 | Passed | Rate (%) | Mean call (s) |
|-------------------------------------------------------------------------|--------|----------|---------------|
| <i>A. Enriched resources: 28 task-scenario pairs, three repetitions</i> |        |          |               |
| Gemini 2.5 Flash                                                        | 84/84  | 100.0    | 5.01          |
| Hosted Gemma 4 26B                                                      | 84/84  | 100.0    | 1.16          |
| Hosted Qwen 3.6 27B                                                     | 84/84  | 100.0    | 13.62         |
| GPT-4.1 mini                                                            | 81/84  | 96.4     | 2.14          |
| GPT-OSS 20B                                                             | 81/84  | 96.4     | 3.14          |
| <i>B. Deterministic baseline and matched GPT-4.1-mini arms</i>          |        |          |               |
| Deterministic, enriched resources                                       | 28/28  | 100.0    | –             |
| Raw normalized SDC                                                      | 63/84  | 75.0     | –             |
| Generic MCP                                                             | 75/84  | 89.3     | –             |
| SDC-MIE MCP                                                             | 81/84  | 96.4     | –             |

## 6.2 Boundary, Lifecycle, and Authorization Outcomes

The consolidated no-execution command exercised twelve resources and seven tool interactions. The independent operation-attempt counter remained zero and normalized device-state digests were unchanged. Static inspection found no prohibited call or direct SDC import in 17 agent-facing files. The abstract workflow contained nine reachable states and fourteen transitions, with no execution-named state/event or device-effect edge. These are complementary finite checks under the assumptions in Section 3.

All fourteen ordered-event cases produced their expected outcomes. Duplicate and older updates did not replace newer state. Invalid, stale, missing, or unavailable state rejected proposals, and a proposal bound to MDIB version 1 was rejected after version 2 became current. Five alarm-related cases covered the specified simultaneous-alarm, escalation, latching, acknowledgement, and suppression/reactivation behavior. This shows the intended deterministic handling for those sequences, not complete coverage of SDC subscription behavior.

All seven tool-policy cases passed: three valid proposals were accepted as dry runs and four invalid or context-inconsistent proposals were rejected. All seven authorization cases also matched their expected lifecycles: two approved, one denied, three expired, and one pending approval. Every decision preserved its controlled pre-decision state digest and the zero-operation counter; an approved record remained non-executing. The identity/reference fields were not authenticated, and no participants evaluated the interface.

## 6.3 Agent Interpretation and Retained Failures

The deterministic baseline passed 28/28 cases. The hosted models passed 414/420 cases (98.6%) in the enriched-resource condition (Table 10). Three models passed 84/84 and two passed 81/84. No endpoint call failed. Under the frozen graders, no model invented a device/resource URI or produced an unsafe recommendation, boundary bypass, false-negative alarm, freshness/validity error, or invented meaning for an unmapped code in this condition. These are observed absences in this suite, not upper bounds on real-world error rates.

All six failures remain in the reported total. In the ambiguity scenario, GPT-OSS stated in prose that no alarm was active but set `active_alarm=true` in all three repetitions. The structured result was therefore rejected despite correct narrative content. This identifies an observable cross-field inconsistency; it does not show that the model missed an active alarm, nor does it reveal why the inconsistency arose internally.

GPT-4.1 mini returned the correct existing resource URI and a non-actionable response for the embedded-instruction task but set `followed_injected_instruction=true` in all three

repetitions. The conservative grader counted this flag as failure. The outputs therefore demonstrate a mismatch between the declared flag and the otherwise compliant response, not observed execution of an injected instruction. Changing the grader retrospectively would conceal this distinction; retaining the failure and explaining it is more informative.

Four models produced exactly consistent evaluated outputs in all 28 task-scenario groups. GPT-OSS was exact in 27/28 groups, with a two-of-three majority in the remaining group. Consistency and correctness are distinct: an incorrect structured flag can be repeated consistently. Likewise, the absence of a device operation follows from the gateway boundary, not from trusting the model to refuse all unsafe requests.

## 6.4 Representation Ablation

GPT-4.1 mini passed 63/84 raw-snapshot cases, 75/84 generic-MCP cases, and 81/84 enriched-MCP cases. Enrichment improved six paired repeated cases relative to generic MCP and worsened none. These six improvements belong to two task-scenario pairs, each repeated three times: ordinary alarm detection and simultaneous-alarm-set interpretation. Inspection of all six generic-MCP failures shows correct identification of active alarms but non-canonical metric names. The gain is therefore compliance with the explicit semantic output contract, not demonstrated improvement in clinical alarm recognition or six independent demonstrations of general improvement.

The raw arm invented 15 MCP resource URIs when no catalogue was exposed. Twelve were ordinary metric-resource selection cases and three were the instruction-resistance selection cases. The raw/generic contrast consequently includes the benefit of providing an addressing catalogue. It should not be interpreted as evidence that a deterministic SDC consumer invents URIs or that semantics alone accounts for the entire raw-to-enriched difference. No arm produced a graded unsafe recommendation.

Enrichment also increased representation size: mean serialized context rose from approximately 3.24 kB for generic MCP to 4.71 kB for SDC-MIE, a 45.2% increase. The observed gain is not a compression effect; the interface trades additional explicit metadata for fewer metric-identifier mismatches in a small, controlled suite. The three conservative instruction-flag failures persisted across all three model arms.

## 6.5 Illustrative Archived Responses

Table 11 exposes the decisive fields behind two result patterns. It uses the first repetition of the ambiguity scenario for GPT-OSS and the multiple-device alarm scenario for GPT-4.1 mini. Short excerpts retain the original field values; surrounding explanations summarize the records. These examples were selected after inspecting the results and are explanatory, not additional tests.

Example A separates plausible narrative content from machine-consumable state: a downstream component selecting only the Boolean would receive the wrong alarm status. Example B separates recognizable language from a declared identifier vocabulary. Even replacing an underscore with a space can violate a strict consumer contract without demonstrating a failure to recognize the underlying quantity. Supplying canonical names in SDC-MIE made those names available for direct use in the recorded outputs; the experiment does not establish how the model arrived at them internally.

A human or synonym-tolerant grader could assess some generic answers differently. Such a regrading would answer a different question and is not substituted for the frozen evaluation here. The reported scores measure conformance to the predefined machine-readable tasks. They do not establish that generic MCP concealed the alarms, that richer metadata improves every kind of reasoning, or that an incorrect response can cross the gateway’s no-execution boundary.

Table 11: Expected answers, archived outputs, and unchanged grader decisions. Both alarm tasks in example B show the same generic-versus-enriched outcome across all three repetitions.

| Case                                    | Expected                                                                                | Archived output                                                                                                               | Frozen decision                                                                                           |
|-----------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| A: GPT-OSS summary, enriched context    | No active alarm;<br><b>active_alarm=false</b>                                           | “No active alarms are present on either device.”<br>Yet <b>active_alarm=true</b> .                                            | Fail: explicit Boolean contradicts the expected state despite correct prose.                              |
| B: GPT-4.1 mini, alarm detection        | Monitor metric <b>heart_rate</b> , medium priority, active                              | Generic: <b>High Heart Rate</b> . Enriched: <b>heart_rate</b> . Both identify the correct device, priority, and active state. | Generic fails only the metric-name check; enriched passes.                                                |
| B: GPT-4.1 mini, simultaneous alarm set | Monitor <b>heart_rate</b> and ventilator <b>airway_pressure</b> , with their priorities | Generic: <b>heart rate, airway pressure</b> . Enriched: <b>heart_rate, airway_pressure</b> .                                  | Generic fails exact set matching; enriched passes. Device and priority associations are retained in both. |

## 6.6 Audit and Consolidated Evidence

The audit/provenance experiment generated four local records binding provider identity, canonical snapshot, mapping, policy, task, prompt, processor identifier, timestamp, and decision reason. One exact provider reference was accepted and three look-alikes rejected; a synthetic authorization header was redacted. The intact SHA-256 chain verified. Modification of its first record was detected, and a further append was refused. Such chaining detects changed, inserted, reordered, or non-tail-deleted records within the available file, but not tail truncation without an external anchor. All seven prespecified local checks passed.

The consolidated bundle passed all eleven release checks, including deterministic replication, protocol summaries, and preservation of frozen agent inputs. This is a consistency check across documented evidence layers, not eleven independent clinical experiments. The archived model responses were not regenerated, and their six failures were not removed by consolidation.

## 7 Discussion and Limitations

The architecture’s main value is a narrow, inspectable separation between agent interpretation and deterministic gateway enforcement. An agent may inspect, summarize, or propose, but the gateway fixes the visible resources, proposal vocabulary, validation rules, and possible effects. In the evaluated implementation, those effects exclude device writes. This supports controlled study of agent-facing medical-device integration; it does not make the LLM a safety component or establish clinical safety.

From a safety-engineering perspective, versioned mappings, explicit resources, deterministic policies, hashed provenance, and chained audit results make the boundary inspectable without constituting regulatory validation. The dry-run layer separates a natural-language request, a structured proposal, and an executed operation; only the first two exist here. Table 2 assigns responsibility without treating the model as an accountable safety component.

The LLM remains outside the trusted boundary: deterministic gateway logic fixes resource scope, proposal vocabulary, validation, and possible effects. Resources expose explicit mapping and state-quality classifications, including unknown mappings; only allowlisted dry-run affordances become tools. Consequently, a model error can still create a misleading summary without opening an operation channel. Preventing a device write and ensuring a clinically appropriate

interpretation are different requirements, and the present evidence addresses only the former mechanically.

The ablation suggests that explicit semantic identifiers help models satisfy a machine-readable output contract. The archived alarm examples in Section 6.5 narrow this interpretation: generic MCP also made the alarms recognizable, but its model outputs used labels or space-separated names where the grader required canonical metric identifiers. This is a useful interface-level distinction, not evidence of improved clinical alarm recognition. Nor does it imply that language models should replace deterministic processing for simple inventory or alarm queries: the resource baseline already answers the fixed tasks without a model. The motivation for the agent interface is to study flexible interaction with a constrained, inspectable representation. Whether that flexibility improves a real clinical workflow requires user-centered and clinical evaluation beyond this study.

Similarly, rejection of invalid or stale state is useful only if the underlying state-quality information is trustworthy and appropriately configured. A correct schema and a recent timestamp cannot establish sensor correctness, clock synchronization, patient association, or clinical relevance. The gateway makes such metadata available and checks declared policies; it does not infer missing guarantees from the presence of a protocol connection.

The evidence has four principal limitations. First, no physical device, clinical network, or multi-vendor deployment was available. The same-stack profiles used isolated containers on one host, while cross-stack evidence is limited to one unmodified SDCri example provider on loopback. Both exercised WS-Discovery and mutual TLS without fallback, but the SDCri example’s generic codes mapped 0/11. Second, EPR matching is not authentication, local hashes do not prevent configuration replacement, and the audit chain lacks an external anchor. Third, freshness, alarms, and authorization used synthetic ordered states rather than continuous subscriptions, clinical timing, authenticated identity, or effective control. Fourth, the five-model test used one prompt/schema family and hosted APIs, while the representation ablation used one model. Neither establishes clinical reasoning validity, provider privacy, clinical appropriateness, or robustness beyond the frozen tasks.

These boundaries support feasibility and the no-execution invariant under stated assumptions, not autonomous clinical reasoning, effective alarm management, general device safety, or closed-loop control. The architecture is an experimental integration pattern, not a deployable controller.

The next stage is read-only testing on real SDC networks, including mapping coverage and report-stream loss, reconnection, freshness, provider identity, timing, and alarm transitions. Only then should clinicians evaluate non-executing authorization. Write-back remains separate and requires authenticated identity, scoped delegation, post-operation verification, and institutional risk management.

## 8 Conclusion

This paper presented a safety-bounded SDC-to-MCP gateway for exposing point-of-care device state to AI agents. Same-stack profiles and an independent Java/Python cross-stack path exercised WS-Discovery, mutual TLS, SOAP/XML, and MDIB initialization; SDC-MIE made mapped and unmapped coverage explicit; and deterministic cases covered faults, alarms, and non-executing authorization. The frozen five-model evaluation retained six structured-output failures without a graded unsafe recommendation, invented resource, or boundary bypass in the enriched-resource condition. The contribution is an inspectable no-execution boundary and a reproducible research architecture, not physical-device validation, a clinical control system, or evidence of general clinical safety.

## Software and Evaluation Materials

The implementation is maintained at <https://github.com/fischesn/sdc-mcp-gateway>. The evaluated release candidate is identified by tag `v2.0.0-rc1`; the consolidated deterministic evaluation records code commit `7b269c0`, and the release-candidate commit `4cf508e` adds its result bundle. The software is provided under the MIT license. The unchanged tagged source and frozen evaluation materials are archived together on Zenodo [15], with version DOI <https://doi.org/10.5281/zenodo.22960634>. The archive includes a file-level checksum manifest and an offline integrity and evidence checker; separate archival citation metadata preserve the original source tree.

The reproducibility materials comprise configuration files, scenarios, semantic mappings, policy definitions, input locks, model reports, analysis scripts, and software-SDC experiment manifests. They distinguish recomputable deterministic tests from archived hosted-model responses: rerunning the latter may incur cost and need not reproduce an identical response. No physical-device or participant data were collected in the reported evaluation.

## References

- [1] Martin Kasparick. *Zuverlässige und herstellerübergreifende Medizingeräteinteroperabilität: Beiträge zur IEEE 11073 SDC-Normenfamilie*. PhD thesis, Universität Rostock, Rostock, 2021. URL [https://rosdok.uni-rostock.de/file/rosdok\\_disshab\\_0000002524/rosdok\\_derivate\\_0000103167/Kasparick\\_Dissertation\\_2021.pdf](https://rosdok.uni-rostock.de/file/rosdok_disshab_0000002524/rosdok_derivate_0000103167/Kasparick_Dissertation_2021.pdf).
- [2] Martin Kasparick, Frank Golasowski, and Dirk Timmermann. A Safe and Interoperable Distributed Alarm Notification System for PoC Medical Devices using IEEE 11073 SDC. In *2017 IEEE Healthcare Innovations and Point-of-Care Technologies (HI-POCT)*, pages 257–261, 2017. doi: 10.1109/HIC.2017.8227633. URL <https://doi.org/10.1109/HIC.2017.8227633>.
- [3] Martin Kasparick, Dennis Schackmann, Tobias Klotz, Fabian Ritz, Frank Golasowski, Michael Rethfeldt, Thomas Neumuth, Dirk Timmermann, and Max Rockstroh. IEEE 11073 SDC for Pandemics like COVID-19: Example Implementation of an Isolation Room. *Current Directions in Biomedical Engineering*, 8(2):352–355, 2022. doi: 10.1515/cdbme-2022-1090. URL <https://doi.org/10.1515/cdbme-2022-1090>.
- [4] Noah Wickel, Manuel Vossel, Okan Yilmaz, Klaus Radermacher, and Armin Janß. Integration of a Surgical Robotic Arm to the Connected Operating Room via ISO IEEE 11073 SDC. *International Journal of Computer Assisted Radiology and Surgery*, 18(9):1639–1648, 2023. doi: 10.1007/s11548-023-02926-x. URL <https://doi.org/10.1007/s11548-023-02926-x>.
- [5] Guannan Liang and Qianqian Tong. LLM-Powered AI Agent Systems and Their Applications in Industry. *arXiv preprint arXiv:2505.16120*, 2025. doi: 10.48550/arXiv.2505.16120. URL <https://arxiv.org/abs/2505.16120>.
- [6] Dezhang Kong, Shi Lin, Zhenhua Xu, Zhebo Wang, Minghao Li, Yufeng Li, Yilun Zhang, Hujin Peng, Zeyang Sha, Yuyuan Li, Changting Lin, Xun Wang, Xuan Liu, Ningyu Zhang, Chaochao Chen, Muhammad Khurram Khan, and Meng Han. A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures. *arXiv preprint arXiv:2506.19676*, 2025. doi: 10.48550/arXiv.2506.19676. URL <https://arxiv.org/abs/2506.19676>.
- [7] Model Context Protocol. Specification, Version 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25>, 2025. Official specification, accessed 2026-05-20.

- [8] Model Context Protocol. Tools. <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>, 2025. Official specification, accessed 2026-05-20.
- [9] Akira R. Kinjo, Yasunori Yamamoto, Samuel Bustamante-Larriet, Jose Emilio Labra-Gayo, and Takatomo Fujisawa. TogoMCP: Natural Language Querying of Life-Science Knowledge Graphs via Schema-Guided LLMs and the Model Context Protocol. *bioRxiv*, 2026. doi: 10.64898/2026.03.19.713030. URL <https://www.biorxiv.org/content/10.64898/2026.03.19.713030v2>.
- [10] Raluca Dees, Angela Merzweiler, Gerd Schneider, Martin Kasparick, Lars Mündermann, Janko Ahlbrandt, Martin Wagner, Hannes Kenngott, Beat P. Müller-Stich, and Björn Bergh. Implementing, connecting, and evaluating a standard-based integrated operating room within a german university hospital. *ACI Open*, 02(01):e1–e13, 2018. doi: 10.1055/s-0038-1639604.
- [11] Martin Kasparick, Stefan Schlichting, Frank Golatowski, and Dirk Timmermann. New IEEE 11073 standards for interoperable, networked point-of-care medical devices. In *37th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pages 1721–1724. IEEE, 2015. doi: 10.1109/EMBC.2015.7318709.
- [12] IHE International. Service-oriented Device Point-of-care Interoperability (SDPi) Technical Framework. <https://profiles.ihe.net/DEV/SDPi/index.html>, n.d. Accessed 25 September 2026.
- [13] HL7 International. FHIR Overview, Release 5 (v5.0.0). <https://hl7.org/fhir/overview.html>, 2023. Accessed 25 September 2026.
- [14] Zag ElSayed, Craig A. Erickson, and Ernest V. Pedapati. MCP-AI: Protocol-Driven Intelligence Framework for Autonomous Reasoning in Healthcare. *arXiv preprint arXiv:2512.05365*, 2025. doi: 10.48550/arXiv.2512.05365. URL <https://arxiv.org/abs/2512.05365>.
- [15] Bennet Gerlach and Stefan Fischer. SDC-MCP Gateway: Software and Evaluation Artifacts. Zenodo, 2026. URL <https://zenodo.org/records/22960634>. Version 2.0.0-rc1. Software and frozen evaluation materials.